TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A LAB REPORT
ON
Artificial Neural Networks: McCulloch-Pitts, Adaline and Backpropagation

SUBMITTED BY:

Name: Aditya Kumar Shah

Roll No.: 080BCT011

Lab No.: 07

SUBMITTED TO:

Department of Electronics
and Computer Engineering

1. Objectives

1. To design a McCulloch-Pitts neuron trained with Adaline (delta rule) learning that behaves as a 2-input AND gate, and to study the effect of its learning parameters.
2. To extend the same Adaline learning procedure to OR, NAND and NOR gates and to draw the resulting neural nets.
3. To repeat the Adaline experiments using the bipolar ($-1/1$) encoding and compare it with the unipolar ($0/1$) encoding.
4. To implement a McCulloch-Pitts network for the XOR function, which is not linearly separable, and to state the formulas used.
5. To implement and train a multilayer perceptron (MLP) for XOR using the backpropagation algorithm.

2. Theoretical Background

An Adaline (Adaptive Linear Neuron) unit computes a net input $y\_in = b + \sum_i x_i w_i$ and is trained with the delta rule (least mean squares / Widrow-Hoff rule):

$$b(\text{new}) = b(\text{old}) + \alpha(t - y\_in), \qquad w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y\_in)x_i.$$

Training stops when the largest weight/bias change in an epoch falls below a tolerance, or a maximum number of epochs is reached. After training, the class is decided by a threshold applied to y_in: for the unipolar case the unit fires ($y = 1$) if $y\_in > 0.5$, and for the bipolar case it fires ($y = 1$) if $y\_in \geq 0$, otherwise $y = -1$.

A single Adaline/perceptron unit can only realize linearly separable functions. AND, OR, NAND and NOR are linearly separable and can therefore be learned directly. XOR is not linearly separable, so it requires either a fixed-weight McCulloch-Pitts network built from simpler gates, or a multilayer network with a hidden layer trained by backpropagation, as also noted in the lab sheet.

For the backpropagation algorithm, with sigmoid activation $f(x) = 1/(1 + e^{-x})$ (so $f'(x) = f(x)(1 - f(x))$), each pattern is used to update the weights as follows:

$$\delta_k = (t_k - y_k) f'(y\_in_k), \qquad \delta\_in_j = \sum_k \delta_k w_{jk}, \qquad \delta_j = \delta\_in_j \, f'(z\_in_j),$$

$$\Delta w_{jk} = \alpha \delta_k z_j, \qquad \Delta w_{ij} = \alpha \delta_j x_i, \qquad w(\text{new}) = w(\text{old}) + \Delta w.$$

3. Assignment 1 — AND Gate using Adaline (Unipolar)

A single Adaline unit with two inputs and a bias was trained on the unipolar AND truth table using the delta-rule update above, with learning rate $\alpha = 0.1$, small random initial weights, and a stopping tolerance of 10^{-3} on the largest weight change (run for a maximum of 500 epochs). The unit converged to the correct classification for all four patterns.

Gate	w_1	w_2	b	**Epochs to correct classification**
AND (unipolar)	0.556	0.528	−0.278	2

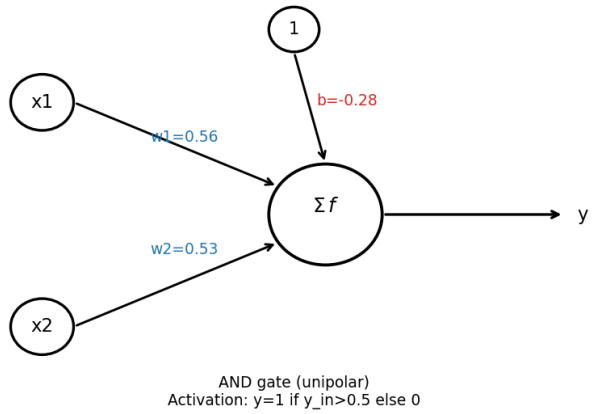

Figure 1: McCulloch-Pitts/Adaline neuron trained as an AND gate (unipolar), with final weights and bias.

3.1 Parameter Variation

To study the effect of the learning rate and the initial weight scale on convergence speed, the AND gate was retrained for $\alpha \in \{0.01, 0.05, 0.1, 0.3, 0.5, 0.9\}$ and for initial-weight scales in $\{0.1, 0.5, 1.0, 2.0\}$, each averaged over five random seeds. The number of epochs needed before the network first classifies all four patterns correctly was recorded.

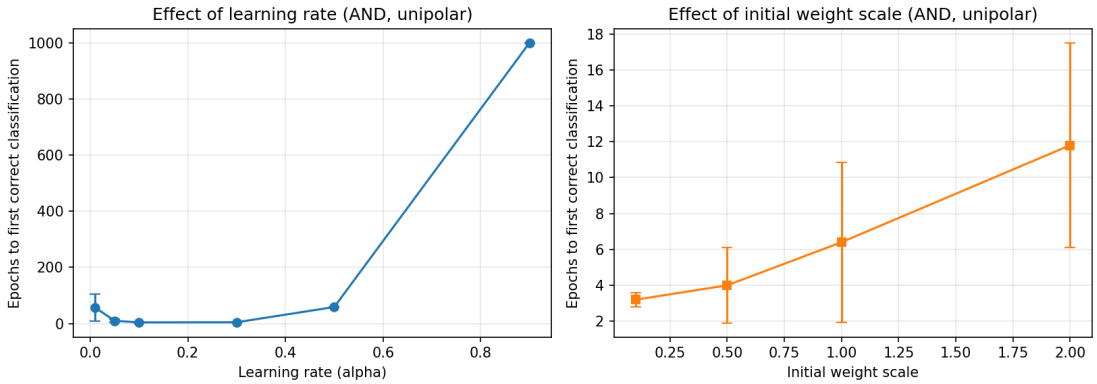

Figure 2: Effect of learning rate and initial weight scale on convergence speed for the AND gate (unipolar Adaline).

The results show a clear U-shaped dependence on the learning rate: a very small rate ($\alpha = 0.01$) is slow because the weight updates are tiny (about 57 epochs on average); a moderate rate ($\alpha \approx 0.1$–0.3) converges fastest (about 4 epochs); and a large rate ($\alpha = 0.9$) makes the weight updates overshoot so much that the SSE oscillates and the unit never settles into the correct classification within 1000 epochs. Larger initial weight magnitudes also slow convergence slightly, since the starting point is farther from the region where all four patterns are already classified correctly.

4. Assignment 2 — OR, NAND and NOR Gates using Adaline

The same Adaline trainer (delta rule, $\alpha = 0.1$, unipolar encoding) was reused for OR, NAND and NOR by only changing the target vector.

Gate	w_1	w_2	b	Epochs to correct classification
OR	0.444	0.472	0.278	4
NAND	−0.556	−0.528	1.278	12
NOR	−0.444	−0.472	0.722	22

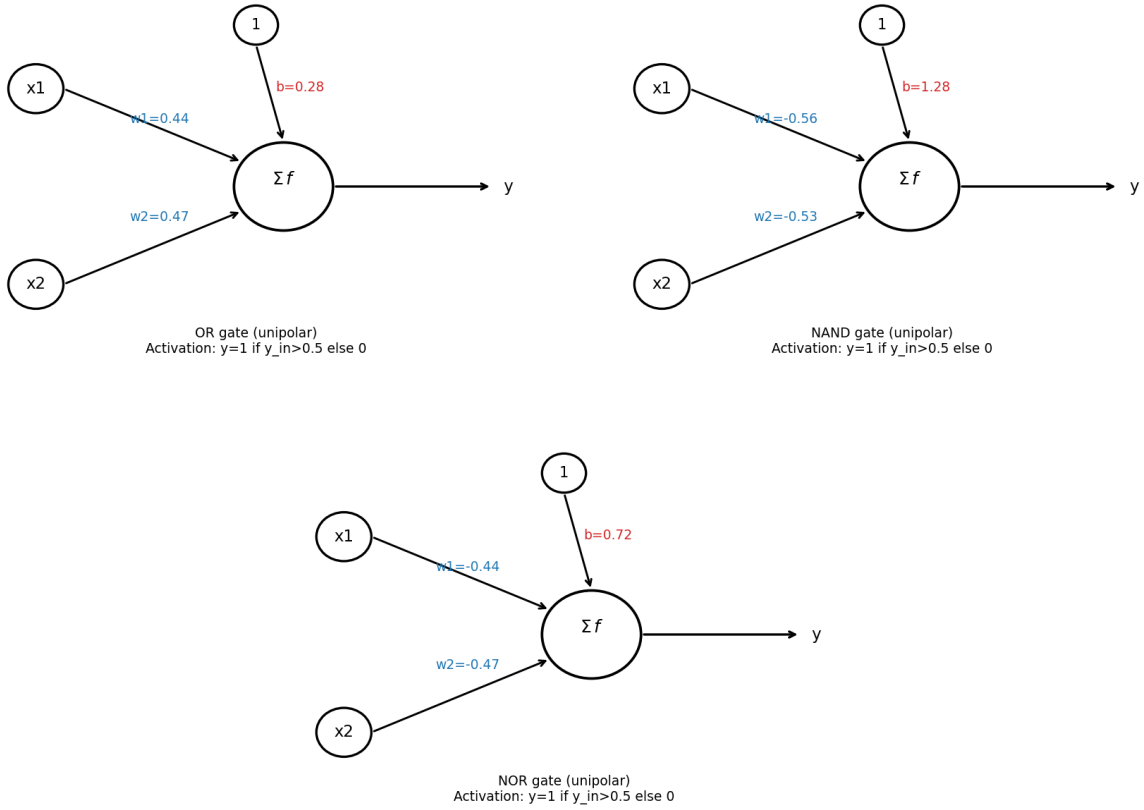

Figure 3: Adaline neurons trained as OR, NAND and NOR gates (unipolar), with final weights and bias.

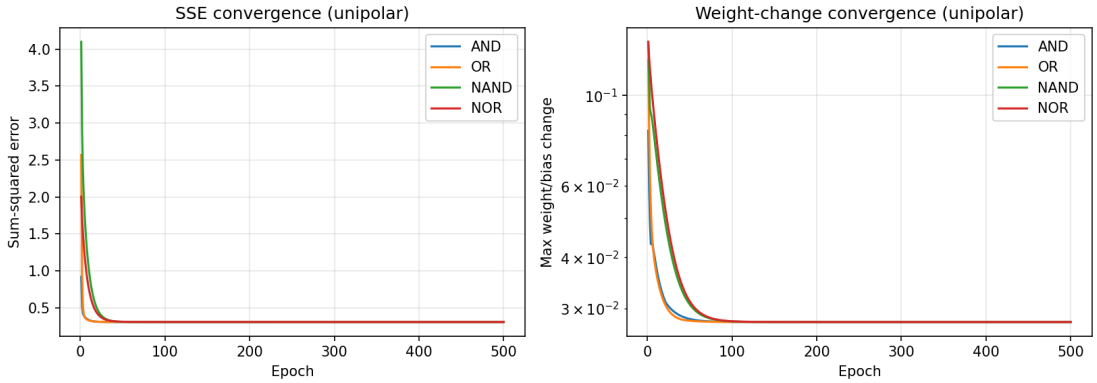

Figure 4: Sum-squared error and weight-change convergence for AND, OR, NAND and NOR (unipolar Adaline, $\alpha = 0.1$).

NAND and NOR take visibly more epochs to reach the correct classification than AND and OR (12 and 22 epochs versus 2 and 4). This happens because NAND/NOR require the bias to dominate the net input for three of the four patterns (i.e. the decision boundary sits close to a corner of the input square), so the delta rule needs more updates to push the bias far enough from its small random initial value.

5. Assignment 3 — Bipolar Model

The AND, OR, NAND and NOR experiments were repeated with the bipolar $(-1/1)$ encoding for both inputs and targets, using the same learning rate and stopping rule.

Gate	w_1	w_2	b	**Epochs to correct classification**
AND	0.529	0.559	−0.5	1
OR	0.471	0.441	0.5	2
NAND	−0.529	−0.559	0.5	2
NOR	−0.471	−0.441	−0.5	3

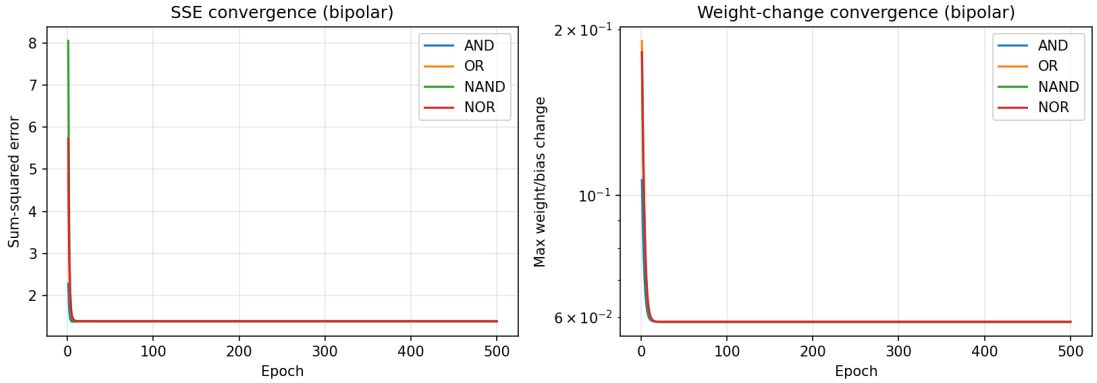

Figure 5: Sum-squared error and weight-change convergence for AND, OR, NAND and NOR (bipolar Adaline, $\alpha = 0.1$).

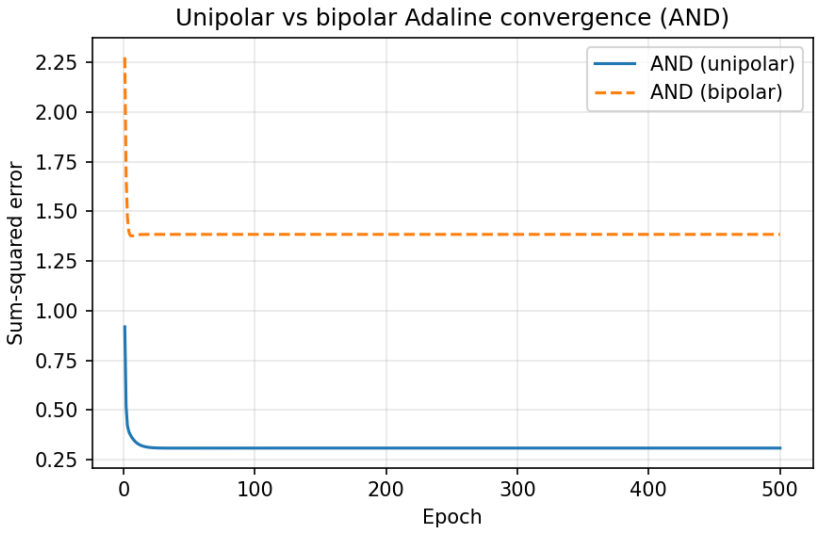

Figure 6: Unipolar vs. bipolar convergence for the AND gate.

Every gate converges markedly faster with the bipolar encoding (1–3 epochs) than with the unipolar encoding (2–22 epochs). The reason is that unipolar inputs use 0 to represent "off", so the weight update $\Delta w_i = \alpha(t - y\_in)x_i$ is exactly zero for any input feature that is 0 in a given pattern – that weight only learns from the patterns where the corresponding input is active. In the bipolar encoding, -1 is a nonzero input, so every pattern updates every weight, and the delta rule pushes the decision boundary into place in far fewer epochs.

6. Assignment 4 — McCulloch-Pitts Network for XOR

XOR is not linearly separable, so a single Adaline/perceptron unit cannot realize it (as noted in the lab sheet). A correct McCulloch-Pitts network can, however, be built by hand from two linearly separable gates in a hidden layer, combined by an output gate:

$$\mathrm{XOR}(x_1, x_2) = \mathrm{AND}\big(\mathrm{OR}(x_1, x_2),\ \mathrm{NAND}(x_1, x_2)\big).$$

Each unit is a binary threshold (McCulloch-Pitts) unit, $y = 1$ if $y\_in \geq \theta$ else 0, with $y\_in = \sum_i w_i x_i$. The following fixed (hand-designed, not learned) weights and thresholds were used:

$$z_1 = \mathrm{OR}(x_1, x_2):\ w = (1, 1),\ \theta_1 = 1 \qquad z_2 = \mathrm{NAND}(x_1, x_2):\ w = (-1, -1),\ \theta_2 = -1$$

$$y = \mathrm{AND}(z_1, z_2):\ w = (1, 1),\ \theta = 2$$

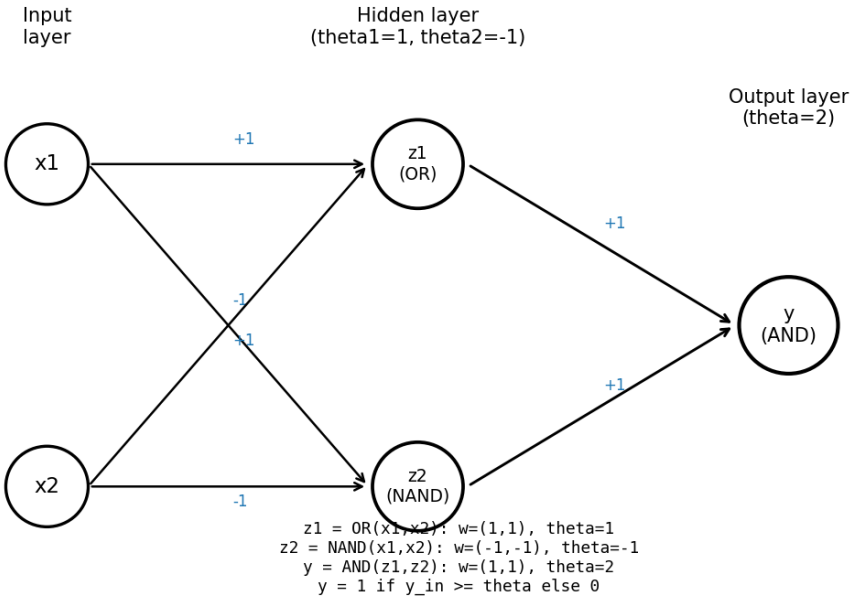

Figure 7: McCulloch-Pitts network for XOR, built from OR and NAND hidden units combined by an AND output unit.

x_1	x_2	z_1 (OR)	z_2 (NAND)	y (AND)	Expected XOR
0	0	0	1	0	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	1	0	0	0

The network reproduces the XOR truth table exactly for all four input patterns.

7. Assignment 5 — MLP for XOR using Backpropagation

A 2-2-1 multilayer perceptron (two inputs, two hidden units, one output unit, all with sigmoid activation and a bias) was trained on the unipolar XOR truth table using the backpropagation formulas of Section 2, with $\alpha = 0.5$. Random weight initialization matters for XOR: of 20 random seeds tried, 13 converged to the correct classification while 7 became stuck in a local minimum with mean-squared error around 0.13 – a well-known difficulty of training small MLPs on XOR with gradient descent. The best converged run (seed 1) was trained for 20000 epochs, reaching a final MSE of about 1.3×10^{-4}.

x_1	x_2	Raw output	Predicted
0	0	0.012	0
0	1	0.987	1
1	0	0.989	1
1	1	0.010	0

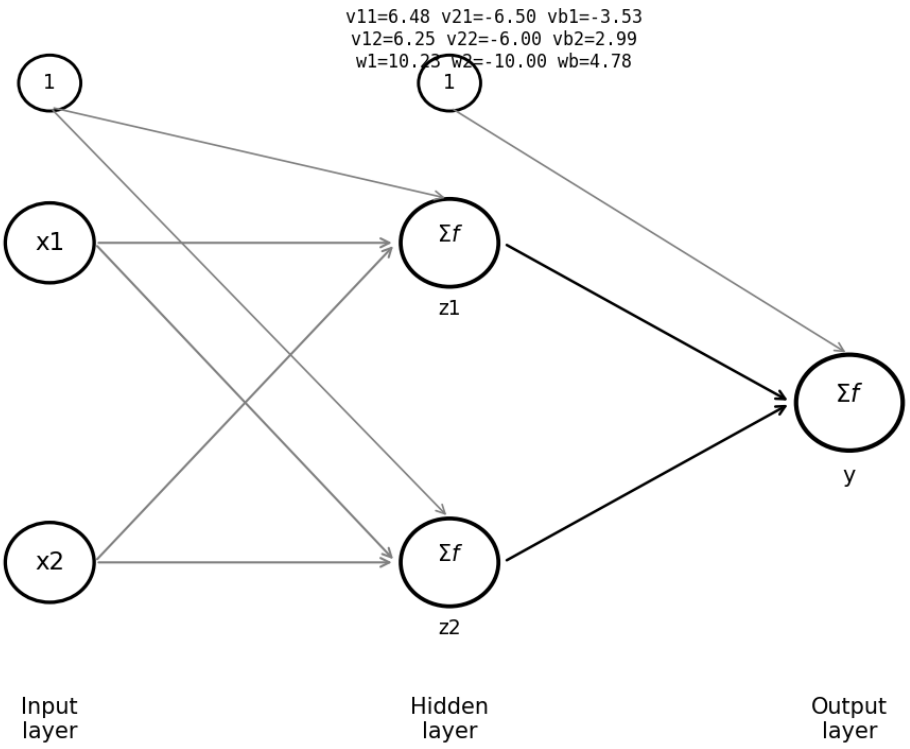

Figure 8: MLP architecture used for XOR, annotated with the final learned weights and biases (v: input-to-hidden, w: hidden-to-output).

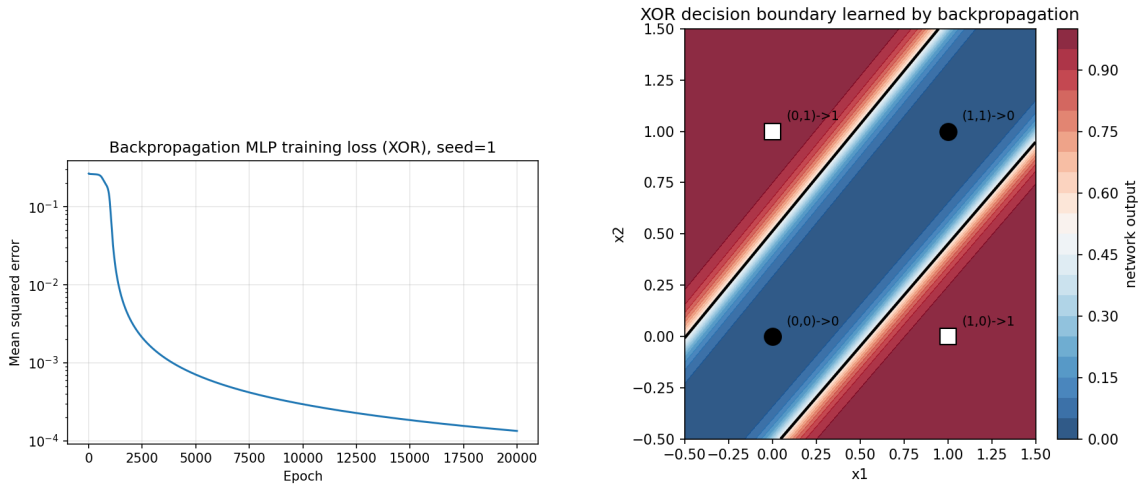

Figure 9: Training loss (mean-squared error, log scale) and the learned decision boundary in input space. The boundary correctly separates the two same-class diagonal corners (0,0) and (1,1) from the two opposite-class corners (0,1) and (1,0), something no single straight line achieves for all four points simultaneously with one hidden layer of two units combining two half-planes.

The loss curve shows the characteristic behaviour of XOR training with sigmoid units: an initial slow plateau while the hidden units' decision boundaries are still poorly placed,

followed by a rapid drop once the two hidden units align to carve out the diagonal band separating the two classes.

8. Conclusion

A single Adaline unit trained with the delta rule successfully learns AND, OR, NAND and NOR under both the unipolar and bipolar encodings, since all four functions are linearly separable; the bipolar encoding converges substantially faster because it lets every input feature contribute to every weight update. The learning rate has a strong, non-monotonic effect on convergence speed: too small is slow, too large causes oscillation and failure to converge, and a moderate rate is best. XOR cannot be realized by a single such unit; it was solved in two different ways: a hand-designed McCulloch-Pitts network combining OR and NAND hidden units through an AND output unit, and a 2-2-1 MLP trained end-to-end with the backpropagation algorithm, which reached the same input-output mapping purely from data despite occasionally converging to a local minimum depending on the random initial weights.

The Python implementation, plots, JSON result summaries, and diagrams are included in the deliverables folder. The executable source is included in the `code` folder.

Appendix: Source Code

The core implementation is plain NumPy, no ML libraries were used, so the update rules are visible directly in the code. The plotting/diagram scripts (`make_plots.py`, `draw_diagrams.py`, `make_backprop_plots.py`) are not listed here since they are just matplotlib boilerplate for the figures above; they are included in the `code` folder.

A.1 adaline.py — Assignments 1, 2, 3

```python
# Adaline (delta rule) for AND/OR/NAND/NOR gates
# works for both unipolar (0/1) and bipolar (-1/1) encoding

import json
import os
import numpy as np

GATES_UNIPOLAR = {
    "AND":  {"X": np.array([[0,0],[0,1],[1,0],[1,1]]), "t": np.array([0,0,0,1])},
    "OR":   {"X": np.array([[0,0],[0,1],[1,0],[1,1]]), "t": np.array([0,1,1,1])},
    "NAND": {"X": np.array([[0,0],[0,1],[1,0],[1,1]]), "t": np.array([1,1,1,0])},
    "NOR":  {"X": np.array([[0,0],[0,1],[1,0],[1,1]]), "t": np.array([1,0,0,0])},
}

GATES_BIPOLAR = {
    "AND":  {"X": np.array([[-1,-1],[-1,1],[1,-1],[1,1]]), "t": np.array([-1,-1,-1,1])},
    "OR":   {"X": np.array([[-1,-1],[-1,1],[1,-1],[1,1]]), "t": np.array([-1,1,1,1])},
    "NAND": {"X": np.array([[-1,-1],[-1,1],[1,-1],[1,1]]), "t": np.array([1,1,1,-1])},
    "NOR":  {"X": np.array([[-1,-1],[-1,1],[1,-1],[1,1]]), "t": np.array([1,-1,-1,-1])},
}


class Adaline:
    def __init__(self, n_inputs, alpha=0.1, mode="unipolar", seed=0, w_init_scale=0.5):
        rng = np.random.default_rng(seed)
        self.w = rng.uniform(-w_init_scale, w_init_scale, size=n_inputs)
        self.b = rng.uniform(-w_init_scale, w_init_scale)
        self.alpha = alpha
        self.mode = mode

    def net_input(self, x):
        return self.b + np.dot(self.w, x)

    def activate(self, y_in):
        if self.mode == "unipolar":
            return 1 if y_in > 0.5 else 0
        else:
```

```
            return 1 if y_in >= 0 else -1

    def train(self, X, t, tol=1e-3, max_epochs=500):
        history = {"epoch": [], "sse": [], "max_dw": [], "weights":
            [], "bias": []}
        first_correct_epoch = None

        for epoch in range(1, max_epochs + 1):
            max_dw = 0
            sse = 0
            for xi, ti in zip(X, t):
                y_in = self.net_input(xi)
                error = ti - y_in
                sse += error ** 2

                # delta rule (Widrow-Hoff) update
                dw = self.alpha * error * xi
                db = self.alpha * error
                self.w += dw
                self.b += db

                max_dw = max(max_dw, np.max(np.abs(dw)), abs(db))

            history["epoch"].append(epoch)
            history["sse"].append(float(sse))
            history["max_dw"].append(float(max_dw))
            history["weights"].append(self.w.tolist())
            history["bias"].append(float(self.b))

            if first_correct_epoch is None and np.array_equal(self.
                predict(X), t):
                first_correct_epoch = epoch

            if max_dw < tol:
                break

        history["first_correct_epoch"] = first_correct_epoch
        return history

    def predict(self, X):
        return np.array([self.activate(self.net_input(xi)) for xi in
            X])


def run_gate(gate_name, mode, alpha=0.1, seed=0, tol=1e-3, max_epochs
    =500, w_init_scale=0.5):
    table = GATES_UNIPOLAR if mode == "unipolar" else GATES_BIPOLAR
    X, t = table[gate_name]["X"], table[gate_name]["t"]

    model = Adaline(2, alpha=alpha, mode=mode, seed=seed,
        w_init_scale=w_init_scale)
    history = model.train(X, t, tol=tol, max_epochs=max_epochs)
    preds = model.predict(X)

    return {
        "gate": gate_name,
        "mode": mode,
        "alpha": alpha,
```

```
        "seed": seed,
        "tol": tol,
        "epochs_run": history["epoch"][-1],
        "final_weights": model.w.tolist(),
        "final_bias": model.b,
        "predictions": preds.tolist(),
        "targets": t.tolist(),
        "converged_to_target": bool(np.array_equal(preds, t)),
        "first_correct_epoch": history["first_correct_epoch"],
        "history": history,
    }


if __name__ == "__main__":
    outdir = os.path.join(os.path.dirname(__file__), "..", "outputs")
    os.makedirs(outdir, exist_ok=True)

    all_results = {"unipolar": {}, "bipolar": {}}

    for gate in ["AND", "OR", "NAND", "NOR"]:
        res = run_gate(gate, "unipolar", alpha=0.1, seed=1)
        all_results["unipolar"][gate] = res
        print(f"[unipolar] {gate}: epochs={res['epochs_run']}, w={np.
            round(res['final_weights'],4)}, "
              f"b={round(res['final_bias'],4)}, preds={res['
                  predictions']}, converged={res['converged_to_target
                  ']}")

    for gate in ["AND", "OR", "NAND", "NOR"]:
        res = run_gate(gate, "bipolar", alpha=0.1, seed=1)
        all_results["bipolar"][gate] = res
        print(f"[bipolar] {gate}: epochs={res['epochs_run']}, w={np.
            round(res['final_weights'],4)}, "
              f"b={round(res['final_bias'],4)}, preds={res['
                  predictions']}, converged={res['converged_to_target
                  ']}")

    with open(os.path.join(outdir, "adaline_results.json"), "w") as f
        :
        json.dump(all_results, f, indent=2)

    print("saved outputs/adaline_results.json")
```

A.2 param_study.py — Assignment 1 (parameter variation)

```
# parameter variation study for the AND gate (assignment 1)
# checks how learning rate and initial weight scale affect
    convergence speed

import json
import os
import numpy as np
from adaline import run_gate

outdir = os.path.join(os.path.dirname(__file__), "..", "outputs")
os.makedirs(outdir, exist_ok=True)
```

```
11
12 alphas = [0.01, 0.05, 0.1, 0.3, 0.5, 0.9]
13 seeds = [1, 2, 3, 4, 5]
14
15 alpha_rows = []
16 for a in alphas:
17     epochs_list = []
18     for s in seeds:
19         res = run_gate("AND", "unipolar", alpha=a, seed=s, tol=1e-3,
               max_epochs=1000)
20         fce = res["first_correct_epoch"] if res["first_correct_epoch"
               ] is not None else 1000
21         epochs_list.append(fce)
22     alpha_rows.append({
23         "alpha": a,
24         "mean_epochs_to_correct": float(np.mean(epochs_list)),
25         "std_epochs_to_correct": float(np.std(epochs_list)),
26         "runs": epochs_list,
27     })
28     print(f"alpha={a}: mean epochs to correct = {np.mean(epochs_list)
           :.2f}")
29
30 w_scales = [0.1, 0.5, 1.0, 2.0]
31 wscale_rows = []
32 for ws in w_scales:
33     epochs_list = []
34     for s in seeds:
35         res = run_gate("AND", "unipolar", alpha=0.1, seed=s, tol=1e
               -3, max_epochs=1000, w_init_scale=ws)
36         fce = res["first_correct_epoch"] if res["first_correct_epoch"
               ] is not None else 1000
37         epochs_list.append(fce)
38     wscale_rows.append({
39         "w_init_scale": ws,
40         "mean_epochs_to_correct": float(np.mean(epochs_list)),
41         "std_epochs_to_correct": float(np.std(epochs_list)),
42         "runs": epochs_list,
43     })
44     print(f"w_init_scale={ws}: mean epochs to correct = {np.mean(
           epochs_list):.2f}")
45
46 with open(os.path.join(outdir, "param_study.json"), "w") as f:
47     json.dump({"alpha_study": alpha_rows, "w_scale_study":
           wscale_rows}, f, indent=2)
48
49 print("saved outputs/param_study.json")
```

A.3 mcp_xor.py — Assignment 4

```
1 # hand-designed McCulloch-Pitts net for XOR (assignment 4)
2 # single perceptron can't do XOR since it's not linearly separable,
      so we
3 # build it from two gates in a hidden layer: XOR = AND(OR(x1,x2),
      NAND(x1,x2))
4
5 import json
```

```python
import os
import numpy as np

outdir = os.path.join(os.path.dirname(__file__), "..", "outputs")
os.makedirs(outdir, exist_ok=True)


def mp_unit(x, w, theta):
    y_in = float(np.dot(w, x))
    return (1 if y_in >= theta else 0), y_in


Z1_W, Z1_T = np.array([1, 1]), 1       # OR
Z2_W, Z2_T = np.array([-1, -1]), -1    # NAND
Y_W, Y_T = np.array([1, 1]), 2         # AND


def xor_mcp(x1, x2):
    x = np.array([x1, x2])
    z1, z1_in = mp_unit(x, Z1_W, Z1_T)
    z2, z2_in = mp_unit(x, Z2_W, Z2_T)
    y, y_in = mp_unit(np.array([z1, z2]), Y_W, Y_T)
    return {"x1": x1, "x2": x2, "z1_in": z1_in, "z1": z1,
            "z2_in": z2_in, "z2": z2, "y_in": y_in, "y": y}


if __name__ == "__main__":
    expected = {(0, 0): 0, (0, 1): 1, (1, 0): 1, (1, 1): 0}
    rows = [xor_mcp(a, b) for a in (0, 1) for b in (0, 1)]

    all_correct = True
    for r in rows:
        exp = expected[(r["x1"], r["x2"])]
        ok = r["y"] == exp
        all_correct &= ok
        print(f"x1={r['x1']} x2={r['x2']} -> z1(OR)={r['z1']} z2(NAND)={r['z2']} "
              f"-> y={r['y']} expected={exp} {'ok' if ok else 'WRONG'}")

    print("all correct:", all_correct)

    with open(os.path.join(outdir, "mcp_xor_results.json"), "w") as f:
        json.dump({
            "weights": {
                "z1_OR": {"w": Z1_W.tolist(), "theta": Z1_T},
                "z2_NAND": {"w": Z2_W.tolist(), "theta": Z2_T},
                "y_AND": {"w": Y_W.tolist(), "theta": Y_T},
            },
            "truth_table": rows,
            "all_correct": bool(all_correct),
        }, f, indent=2)

    print("saved outputs/mcp_xor_results.json")
```

A.4 backprop_xor.py — Assignment 5

```python
# 2-2-1 MLP trained on XOR using backpropagation (assignment 5)
# sigmoid activation on hidden and output layer

import json
import os
import numpy as np

outdir = os.path.join(os.path.dirname(__file__), "..", "outputs")
os.makedirs(outdir, exist_ok=True)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
T = np.array([0, 1, 1, 0], dtype=float)


def sigmoid(x):
    return 1 / (1 + np.exp(-x))


class MLP_XOR:
    def __init__(self, n_hidden=2, alpha=0.5, seed=0):
        rng = np.random.default_rng(seed)
        self.V = rng.uniform(-1, 1, size=(2, n_hidden))    # input -> hidden weights
        self.vb = rng.uniform(-1, 1, size=n_hidden)          # hidden bias
        self.W = rng.uniform(-1, 1, size=n_hidden)           # hidden -> output weights
        self.wb = float(rng.uniform(-1, 1))                  # output bias
        self.alpha = alpha

    def forward(self, x):
        z_in = self.vb + x @ self.V
        z = sigmoid(z_in)
        y_in = self.wb + z @ self.W
        y = sigmoid(y_in)
        return z_in, z, y_in, y

    def train(self, X, T, max_epochs=20000, tol=1e-4):
        history = {"epoch": [], "mse": []}
        for epoch in range(1, max_epochs + 1):
            mse = 0
            for x, t in zip(X, T):
                z_in, z, y_in, y = self.forward(x)
                mse += (t - y) ** 2

                # error term at output, then propagate back to hidden layer
                d_k = (t - y) * y * (1 - y)
                d_j = d_k * self.W * z * (1 - z)

                # weight updates
                self.W += self.alpha * d_k * z
                self.wb += self.alpha * d_k
                self.V += self.alpha * np.outer(x, d_j)
                self.vb += self.alpha * d_j
```

```
52
53              mse /= len(X)
54              history["epoch"].append(epoch)
55              history["mse"].append(float(mse))
56              if mse < tol:
57                  break
58          return history
59
60      def predict(self, X):
61          outs = np.array([self.forward(x)[3] for x in X])
62          return outs, (outs > 0.5).astype(int)
63
64
65  if __name__ == "__main__":
66      best = None
67      for seed in range(20):
68          model = MLP_XOR(n_hidden=2, alpha=0.5, seed=seed)
69          history = model.train(X, T)
70          outs, preds = model.predict(X)
71          converged = np.array_equal(preds, T.astype(int))
72          if converged and (best is None or history["epoch"][-1] < best
                ["history"]["epoch"][-1]):
73              best = {"seed": seed, "model": model, "history": history,
                    "outs": outs, "preds": preds}
74          print(f"seed={seed}: epochs={history['epoch'][-1]}, mse={
                history['mse'][-1]:.6f}, "
75                f"preds={preds.tolist()}, converged={converged}")
76
77      model, history = best["model"], best["history"]
78      outs, preds = best["outs"], best["preds"]
79      print("\nbest run: seed =", best["seed"])
80      for x, t, o, p in zip(X, T, outs, preds):
81          print(f"x={x.tolist()} target={t} output={o:.4f} predicted={p
                }")
82
83      result = {
84          "seed": best["seed"],
85          "alpha": model.alpha,
86          "epochs_run": history["epoch"][-1],
87          "final_mse": history["mse"][-1],
88          "V_input_hidden": model.V.tolist(),
89          "vb_hidden_bias": model.vb.tolist(),
90          "W_hidden_output": model.W.tolist(),
91          "wb_output_bias": model.wb,
92          "truth_table": [
93              {"x1": int(x[0]), "x2": int(x[1]), "target": int(t), "
                    raw_output": float(o), "predicted": int(p)}
94              for x, t, o, p in zip(X, T, outs, preds)
95          ],
96          "history": history,
97      }
98      with open(os.path.join(outdir, "backprop_xor_results.json"), "w")
            as f:
99          json.dump(result, f, indent=2)
100     print("saved outputs/backprop_xor_results.json")
```